

Análisis forense informático

Tarea 4: Realización de análisis forenses en IoT

Índice

Caso práctico.....	2
Apartado 1: Mapa de la red IoT.....	3
Identificación.....	3
Dispositivos legítimos y no autorizados.....	4
Apartado 2: Análisis de la comunicación MQTT.....	5
Listado de topics.....	5
Obtención de Client ID y topic.....	6
Apartado 3: Análisis de la calidad de servicio (QoS).....	6
Niveles de QoS.....	7
Justificación de uso de niveles QoS.....	8
Apartado 4: Detección y análisis de la amenaza.....	9
Identificación de la fuente del comando malicioso.....	9
Identificación de payload.....	10
Evidencia de recepción del comando malicioso.....	11
Apartado 5: Evaluación de vulnerabilidades y recomendaciones.....	12
Evaluación de vulnerabilidades.....	12
Recomendaciones.....	13

Caso práctico

Eres un analista senior en un Centro de Operaciones de Seguridad (SOC) especializado en infraestructuras críticas. La empresa "AgroTech Solutions", que gestiona invernaderos inteligentes de alta tecnología, ha emitido una alerta de seguridad urgente.

Durante la noche, su sistema de monitorización ha registrado un comportamiento anómalo en el Invernadero Experimental N° 7. El sistema de riego se activó de forma inesperada durante una hora, inundando una sección de cultivos sensibles y causando pérdidas significativas. El equipo de operaciones sospecha que el comando de activación no fue generado por su sistema de control central, sino por una fuente no autorizada.

Antes de que pudieran realizar un análisis completo, el protocolo de seguridad aisló la red y generó una captura de tráfico (`greenhouse_incident.pcap`) que contiene la actividad de red justo antes y durante el incidente. Tu misión como experto forense es analizar esta evidencia para determinar el origen del ataque, reconstruir la secuencia de eventos y proporcionar recomendaciones para prevenir futuros incidentes.

Apartado 1: Mapa de la red IoT

Identificación

Identifica las direcciones IP y los Client ID de todos los dispositivos IoT (excluyendo el *broker*) que se conectan a la red.

Para identificar todas las direcciones IP y sus respectivos Client IDs excluyendo al *broker*, lo más eficiente es buscar los paquetes de conexión **MQTT** (*Connect Command*). En estos paquetes, los dispositivos IoT se anuncian al *broker* enviando su IP de origen y su Client ID. Para ello, ejecutamos el siguiente comando:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.msgtype == 1" -T fields -e ip.src  
-e mqtt.clientid | sort | uniq
```

Donde:

- `-r greenhouse_incident.pcap`: Lee el archivo de captura.
- `-Y "mqtt.msgtype == 1"`: Filtra únicamente los mensajes MQTT de tipo *Connect*. Esto es, los clientes conectándose al *broker*.
- `-T fields`: Indica que queremos extraer campos específicos (indicados con `-e`).
- `-e ip.src`: Extrae la dirección IP de origen. En este caso, la del dispositivo IoT.
- `-e mqtt.clientid`: Extrae el Client ID del dispositivo.
- `| sort | uniq`: Ordena los resultados y elimina los duplicados para tener una visión clara de los dispositivos.



```
~/Pr/E/curso-especializacion-ciberseguridad-ti-privado/a/u4/tarea > main -l 722 /Applications/Wireshark.app/Contents/MacOS/tshark  
-r greenhouse_incident.pcap -Y "mqtt.msgtype == 1" -T fields -e ip.src -e mqtt.clientid | sort | uniq  
192.168.10.101 greenhouse-temp-01  
192.168.10.102 greenhouse-soil-01  
192.168.10.103 greenhouse-irrigation-01  
192.168.10.199 unknown-device
```

López Henestroza, José Carlos

Captura del comando ejecutado

Como podemos ver, el resultado es el siguiente:

```
192.168.10.101 greenhouse-temp-01  
192.168.10.102 greenhouse-soil-01  
192.168.10.103 greenhouse-irrigation-01  
192.168.10.199 unknown-device
```

Dispositivos legítimos y no autorizados

¿Cuántos dispositivos IoT legítimos (sensores/actuadores del invernadero) hay y cuántos dispositivos desconocidos o no autorizados?

En base a los resultados del [apartado anterior](#), podemos identificar **tres sensores/actuadores legítimos**, los cuales son los siguientes:

192.168.10.101	greenhouse-temp-01
192.168.10.102	greenhouse-soil-01
192.168.10.103	greenhouse-irrigation-01

Como contraparte, identificamos **un dispositivo desconocido**, el cual es el siguiente:

192.168.10.199	unknown-device
----------------	----------------

Apartado 2: Análisis de la comunicación MQTT

Listado de *topics*

Lista todos los *topics* MQTT utilizados en la captura. Para cada *topic*, describe brevemente su propósito (ej. "enviar datos de temperatura", "recibir comandos de riego").

En el protocolo MQTT, la información fluye a través de *topics*, que actúan como canales de mensajería jerárquicos a los que los dispositivos se suscriben o publican. Para extraer una lista única de todos los *topics* utilizados durante la captura, ejecutamos el siguiente comando:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.topic" -T fields -e mqtt.topic |  
sort | uniq
```

Donde:

- `-r greenhouse_incident.pcap`: Lee el archivo de captura.
- `-Y "mqtt.topic"`: Filtra la captura para mostrar solo los paquetes MQTT que contienen el campo *topic*.
- `-T fields -e mqtt.topic`: Extrae la cadena de texto correspondiente al nombre del *topic*.
- `| sort | uniq`: Ordena los resultados y elimina los duplicados para tener una visión clara de los dispositivos.



```
~/Pr/E/curso-especializacion-ciberseguridad-ti-privado/a/u4/tarea > main :1 722 /Applications/Wireshark.app/Contents/MacOS/tshark  
-r greenhouse_incident.pcap -Y "mqtt.topic" -T fields -e mqtt.topic | sort | uniq  
greenhouse/actuators/irrigation/cmd  
greenhouse/actuators/irrigation/status  
greenhouse/sensors/soil  
greenhouse/sensors/temperature  
~/Pr/E/curso-especializacion-ciberseguridad-ti-privado/a/u4/tarea > main :1 722 21:41:50
```

Captura del comando ejecutado

El resultado y el propósito de cada *topic* son los siguientes:

- `greenhouse/actuators/irrigation/cmd`: Recibe comandos de riego.
- `greenhouse/actuators/irrigation/status`: Reporta el estado del sistema de riego.
- `greenhouse/sensors/soil`: Envía datos de humedad del suelo.
- `greenhouse/sensors/temperature`: Envía datos de temperatura.

Obtención de Client ID y topic

Uno de los dispositivos legítimos se suscribe a un *topic* para recibir comandos. ¿Cuál es su Client ID y a qué *topic* se suscribe?

Para vincular ese dispositivo con su Client ID real, tenemos que usar el flujo TCP (`tcp.stream`) como puente. Lo hacemos en dos pasos:

1. Identificamos el número de flujo TCP del mensaje SUBSCRIBE

Ejecutamos este comando para ver a qué flujo TCP pertenece esa suscripción:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.msgtype == 8" -T fields -e tcp.stream -e mqtt.topic
```



Captura del comando ejecutado

2. Buscamos el paquete CONNECT de ese mismo flujo TCP

Ahora que sabemos que el número de flujo es 2, buscamos el mensaje de conexión inicial de esa misma conversación para extraer el Client ID:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.msgtype == 1 and tcp.stream == 2" -T fields -e ip.src -e mqtt.clientid
```



Captura del comando ejecutado

Uniendo la información obtenida, sabemos que el Client ID del dispositivo legítimo que se suscribe al *topic* `greenhouse/actuators/irrigation/cmd` para recibir comandos es `192.168.10.103`.

Apartado 3: Análisis de la calidad de servicio (QoS)

Niveles de QoS

¿Qué niveles de QoS se utilizan en la comunicación? Proporciona un ejemplo de un topic que utilice cada nivel de QoS identificado.

Para descubrir qué niveles de Calidad de Servicio (QoS) se están empleando en la red del invernadero, debemos analizar los mensajes de publicación (PUBLISH, `mqtt.msgtype == 3`).

Para extraer todos los niveles de QoS utilizados y cruzarlos con sus respectivos *topics*, ejecutamos el siguiente comando:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.msgtype == 3" -T fields -e mqtt.qos -e mqtt.topic | sort | uniq
```



Captura del comando ejecutado

Como podemos apreciar, se usan los **niveles 0 y 1**. El nivel 0 indica que el mensaje se envía una sola vez y no requiere confirmación de recepción (ACK) por parte del *broker*. Se usa para datos que no son críticos, como los *topics* relacionados con los sensores (en este caso). Por otra parte, el nivel 1 sí que requiere confirmación de recepción (PUBACK), por lo que si el emisor no recibe dicho paquete, volverá a enviar el mensaje. En este caso, se usa para el *topic* `greenhouse/actuators/irrigation/cmd`, el cual recibe comandos de riego.

Justificación de uso de niveles QoS

¿Por qué crees que se utiliza un nivel de QoS más alto para los comandos enviados al sistema de riego en comparación con los datos de los sensores?

La **telemetría ambiental** se envía con mucha frecuencia. Si un paquete de **QoS 0** se pierde por una pequeña caída de red, el impacto es nulo, ya que el sistema central simplemente recibirá la siguiente lectura unos segundos después. No vale la pena gastar recursos de red en confirmar cada paquete.

Sin embargo, los **comandos de control** son esporádicos y piezas fundamentales del sistema. Si el sistema central envía una orden para apagar el irrigador y el paquete se pierde, el agua seguirá saliendo indefinidamente. El **QoS 1** fuerza que el mensaje llegue al menos una vez al exigir una confirmación (**PUBACK**). Si el emisor no recibe dicha confirmación, retransmitirá la orden de apagado hasta que se asegure de que la válvula la ha recibido.

Apartado 4: Detección y análisis de la amenaza

Identificación de la fuente del comando malicioso

¿Cuál es la dirección IP y el Client ID del dispositivo que envió el comando de activación no autorizado?

Basándonos en la identificación de dispositivos que realizamos al inicio de nuestra investigación, sabemos que los datos del dispositivo atacante son IP 192.168.10.199 y Client ID unknown-device.

Para demostrar irrefutablemente que esta IP fue la que envió el comando de activación no autorizado, cruzamos la IP de origen con los mensajes de publicación (PUBLISH) enviados al *topic* de control. El comando de `tshark` para extraer esta evidencia es el siguiente:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.msgtype == 3 and mqtt.topic === 'greenhouse/actuators/irrigation/cmd'" -T fields -e ip.src -e mqtt.msg
```



Captura del comando ejecutado

Como podemos ver, la IP es 192.168.10.199, la cual coincide con la del dispositivo sospechoso localizado en apartados anteriores.

Identificación de *payload*

¿Qué comando exacto (*payload*) envió este dispositivo sospechoso al sistema de riego?

En el apartado anterior, obtuvimos el mensaje en formato hexadecimal. Al convertirlo a texto ASCII, obtenemos el siguiente JSON en texto plano:

```
{"action":"activate","duration":3600}
```

Este *payload* encaja perfectamente con el caso práctico y explica el comportamiento anómalo que desencadenó la alerta del SOC al inicio de nuestro caso. El atacante inyectó el comando para activar las válvulas de agua haciéndose pasar por el sistema central legítimo. El riego estuvo activo durante 3600 segundos. Es decir, 1 hora exacta.

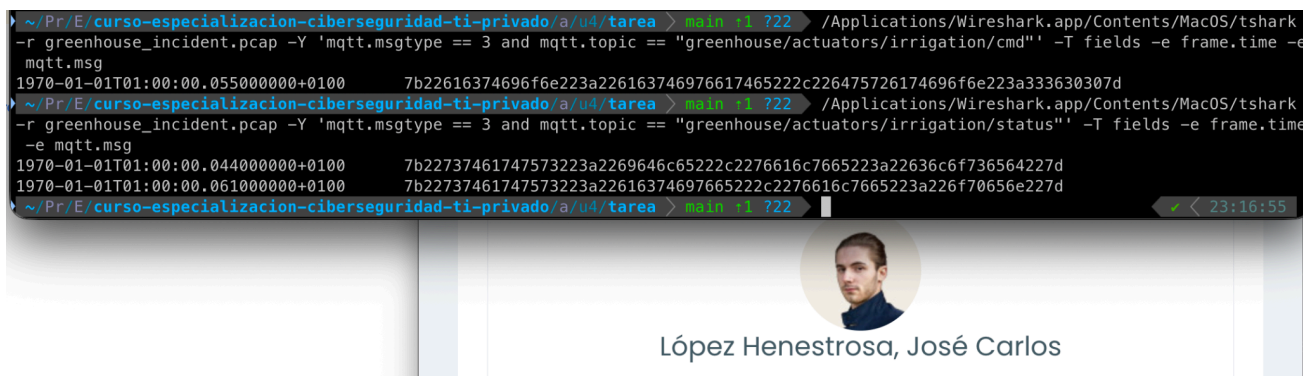
Evidencia de recepción del comando malicioso

¿Cómo confirma la captura de red que el sistema de riego recibió y actuó según el comando malicioso?

Debemos buscar dos tipos de evidencias en la captura de red: la confirmación a nivel de protocolo MQTT y la confirmación a nivel de aplicación.

Por apartados anteriores, sabemos que la arquitectura cuenta con el canal `greenhouse/actuators/irrigation/status`. Cuando un actuador recibe un comando y lo ejecuta físicamente, publica un mensaje en su *topic* de estado para notificar al sistema central que su condición ha cambiado. Para ver si el dispositivo (192.168.10.103) notificó que estaba regando justo después del ataque, ejecutamos este comando buscando en el canal de *status*:

```
tshark -r greenhouse_incident.pcap -Y "mqtt.msgtype == 3 and mqtt.topic === 'greenhouse/actuators/irrigation/status'" -T fields -e frame.time_relative -e mqtt.msg
```



```
~/Pr/E/curso-especializacion-ciberseguridad-ti-privado/a/u4/tarea > main -l 722 /Applications/Wireshark.app/Contents/MacOS/tshark
-r greenhouse_incident.pcap -Y 'mqtt.msgtype == 3 and mqtt.topic == "greenhouse/actuators/irrigation/cmd"' -T fields -e frame.time -e
mqtt.msg
1970-01-01T01:00:00.055000000+0100 7b22616374697666e223a2261637469766617465222c226475726174697666e223a333630307d
~/Pr/E/curso-especializacion-ciberseguridad-ti-privado/a/u4/tarea > main -l 722 /Applications/Wireshark.app/Contents/MacOS/tshark
-r greenhouse_incident.pcap -Y 'mqtt.msgtype == 3 and mqtt.topic == "greenhouse/actuators/irrigation/status"' -T fields -e frame.time
-e mqtt.msg
1970-01-01T01:00:00.044000000+0100 7b22737461747573223a2269646c65222c2276616c7665223a22636c6f736564227d
1970-01-01T01:00:00.061000000+0100 7b22737461747573223a22616374697665222c2276616c7665223a226f70656e227d
~/Pr/E/curso-especializacion-ciberseguridad-ti-privado/a/u4/tarea > main -l 722 23:16:55
```

López Henestroza, José Carlos

Captura del comando ejecutado

Los mensajes de `greenhouse/actuators/irrigation/status` son:

1. **Estado inicial** (T = 0.044s): {"status": "idle", "valve": "closed"}
2. **Estado tras el ataque** (T = 0.061s): {"status": "active", "valve": "open"}

Esto significa que apenas 17 milisegundos después de que el sistema reportara su estado normal y 6 milisegundos después del ataque (se efectuó en T = 0.055s) procesó el comando malicioso del atacante y ejecutó la acción física de abrir el agua.

Apartado 5: Evaluación de vulnerabilidades y recomendaciones

Evaluación de vulnerabilidades

Basado en tu análisis completo, ¿cuál fue la vulnerabilidad de seguridad fundamental en la configuración del *broker* MQTT que permitió que este ataque tuviera éxito?

La vulnerabilidad fundamental que hizo posible este incidente fue la **ausencia total de controles de autenticación y autorización (ACL)** en el *broker* MQTT. Además, también se le suma la comunicación en texto plano.

De forma más extensa, el *broker* permitió que un dispositivo ajeno al invernadero (IP 192.168.10.199, Client ID `unknown-device`) estableciera una conexión sin necesidad de introducir **credenciales ni certificados digitales**. No obstante, incluso si un dispositivo logra conectarse, no debería poder **hablar en cualquier canal**. El *broker* no tenía configuradas políticas de permisos. Permitió que un dispositivo genérico publicara un mensaje en un *topic* reservado para operaciones críticas (`greenhouse/actuators/irrigation/cmd`).

Como pudimos extraer el payload en hexadecimal y traducirlo directamente a JSON, sabemos que la comunicación **no estaba cifrada con TLS/SSL**. Esto facilitó que el atacante pudiera capturar el tráfico previo para aprender la estructura de los *topics* y el formato de los comandos legítimos antes de inyectar el suyo.

Recomendaciones

Como analista forense, ¿qué dos recomendaciones críticas le darías a AgroTech Solutions para asegurar su infraestructura MQTT y prevenir que un incidente similar vuelva a ocurrir?

Para evitar que el invernadero vuelva a ser comprometido, el equipo de ingeniería de AgroTech Solutions debe implementar estas dos medidas de seguridad:

1. Fortalecimiento del *broker* MQTT

Tienen que deshabilitar conexiones anónimas y requerir credenciales únicas para cada sensor o actuador para que solo los dispositivos con *hardware* certificado por la empresa puedan conectarse. Por otro lado, se podrían configurar políticas de lectura/escritura por *topic* para evitar operaciones inesperadas.

2. Cifrado de las comunicaciones

Tienen que configurar MQTT para usar MQTTS (MQTT sobre TLS) para evitar que cualquier atacante en la red local pueda aplicar ingeniería inversa a los comandos leyendo los paquetes en texto plano.